

1)

`clock()`

A função `clock()` mede o tempo de CPU utilizado por um programa desde o seu início, retornando esse valor em “ticks” de processamento. Esse tempo representa apenas o período em que a CPU esteve executando o código, ignorando pausas como espera por entrada/saída ou `sleep`. Para obter o valor em segundos, divide-se o resultado por `CLOCKS_PER_SEC`. É útil principalmente para avaliar o desempenho de algoritmos.

`time()`

A função `time()` retorna o tempo atual do sistema em segundos desde o Epoch Unix (1º de janeiro de 1970, 00:00:00 UTC). Ela fornece uma forma simples de obter o horário atual, mas com baixa precisão, limitada a segundos. É geralmente usada para registros de data e hora, como logs ou marcação de eventos.

`gettimeofday()`

A função `gettimeofday()` obtém o tempo atual com maior precisão, retornando segundos e microssegundos através de uma estrutura (`timeval`). Ela acessa o relógio do sistema e permite medições mais detalhadas de intervalos de tempo. Apesar disso, é considerada obsoleta em sistemas POSIX mais recentes, sendo substituída por alternativas mais modernas.

`clock_gettime()`

A função `clock_gettime()` fornece medições de tempo com alta precisão (até nanosegundos), utilizando diferentes tipos de relógios, como o `CLOCK_REALTIME` (tempo do sistema) e o `CLOCK_MONOTONIC` (tempo contínuo que não sofre ajustes). Ela é amplamente usada em sistemas modernos para medições precisas de desempenho e intervalos de execução.

Ferramenta “perf”

O `perf` é uma ferramenta externa do Linux voltada para análise de desempenho, que utiliza contadores de hardware do processador para medir diversos aspectos da execução de um programa, como ciclos de CPU, instruções executadas e uso de cache. Diferente das funções em C, ele fornece uma visão detalhada e de baixo nível do comportamento do sistema, sendo muito utilizado em otimização avançada.

2)

Algoritmo escolhido: soma dos elementos do array.

3)

Linguagem C:

Com 100 instâncias: máquina 1: 0.000002, máquina 2: 0.000001

Com 1000 instâncias: máquina 1: 0.000004, máquina 2: 0.000002

Com 10000 instâncias: máquina 1: 0.000014, máquina 2: 0.000012

Python:

Com 100 instâncias: máquina 1: 0.0000055100, máquina 2: 0.0000046700

Com 1000 instâncias: máquina 1: 0.0008269400, máquina 2: 0.0000569400

Com 10000 instâncias: máquina 1: 0.0051941290, máquina 2: 0.0008269400

Java:

Com 100 instâncias: máquina 1: 0.072232855, máquina 2: 0.167545562

Com 1000 instâncias: máquina 1: 0.823750646, máquina 2: 0.329764457

Com 10000 instâncias: máquina 1: 3.250554190, máquina 2: 3.257737868

Compilador diferente na mesma máquina:

Ao mudar de compilador na máquina 1 o tempo de execução foi para 0.000001 segundos (metade do tempo). O primeiro compilador utilizado foi o programiz e o segundo foi o OnlineGDB para o C. Em java e python teve a mesma diferença de tempo para compiladores diferentes.

Configuração da máquina 1: Processador i7 9, 16 RAM, Windows, compiladores online programiz e OnlineGDB.

4)

Linguagem	Função usada	Precisão	100 instâncias	1000 instâncias	10000 instâncias	Observações
C	clock()	Média	M1: 0.000002 M2: 0.000001	M1: 0.000004 M2: 0.000002	M1: 0.000014 M2: 0.000012	Extremamente rápido e consistente
Python	perf_counter()	Alta	M1: 0.0000055100 M2: 0.0000046700	M1: 0.0008269400 M2: 0.0000569400	M1: 0.0051941290 M2: 0.0008269400	Grande variação entre máquinas
Java	nanoTime()	Muito alta	M1: 0.072232855 M2: 0.167545562	M1: 0.823750646 M2: 0.329764457	M1: 3.250554190 M2: 3.257737868	Tempo alto por causa da JVM

Parte II:

Contagem de Instruções (Ic): 1.087.222

Ciclos de Clock: 1.275.267

$CPI = 1.275.267 / 1.087.222 \approx 1,17$

Um CPI de 1,17 é considerado muito bom e indica um processador eficiente (ele gastou pouco mais de 1 ciclo por instrução). Olhando para os dados, tivemos cerca de 25 mil cache-misses e 22 mil branch-misses. Olhando esses números isolados eles parecem grandes, mas quando você compara com o total de 1 milhão de instruções, isso representa uma taxa de erro muito baixa (cerca de 2%). Porque o algoritmo (um for sequencial somando um vetor) é extremamente previsível. O "Branch Predictor" do processador adivinha fácil que o loop vai continuar rodando, e o "Prefetcher" da memória cache já vai puxando os próximos números do vetor antes mesmo do seu programa pedir (Localidade Espacial). Portanto, essas faltas não impactaram negativamente o seu CPI.

Parte III:

Benchmarks escolhidos: gzip e mcf.

GZIP: máquina 1: 18.5, máquina 2: 24.2, máquina 3: 14.1

MCF: máquina 1: 42.8, máquina 2: 55.6, máquina 3: 31.7

Média geométrica

$$M1: \sqrt{(1 \times 1)} = 1.00$$

$$M2: \sqrt{(1.31 \times 1.30)} \approx 1.30$$

$$M3: \sqrt{(0.76 \times 0.74)} \approx 0.75$$

Máquina com melhor desempenho: máquina 3.